

Python Full Stack Assignment

CodeHub — Mini User Profile System

Scenario

You are working as a Python developer for a small social-learning platform called **CodeHub**. Before a user can access the application, the backend needs to process and display their profile information correctly.

Your task is to build a **console-based User Profile System** using only the concepts covered so far.

Concepts you MUST use

String literals • Variables • `input()` • f-strings • String concatenation • `.upper()` • `.lower()` • `.title()` • `.capitalize()` • `.swapcase()` • `.casefold()` • `.find()` • `.rfind()` • `.index()` • `.count()` • `.startswith()` • `.endswith()` • `.strip()` • `.lstrip()` • `.rstrip()` • `.split()` • `.join()`

Do not use: string formatting specifiers such as `:.2f`, loops, functions, classes, regular expressions, or collections for solving the individual string operations.

Part 1 — User Registration

Ask the user for their full name, email, phone number, city, favourite technology, and skills separated by commas.

```
Enter your full name: john doe
Enter your email: JOHN.DOE@GMAIL.COM
Enter your phone number: 9876543210
Enter your city: hyderabad
Enter your favourite technology: python
Enter your skills separated by comma: python, django, react, sql
```

Clean the inputs using appropriate methods. Convert the name to title case, city to title case, and email to lowercase.

Part 2 — Generate a Username

Using the user's name and email, generate a username. For `john.doe@gmail.com`, the expected username is `john.doe`. Hint: use `find('@')` and slicing.

Part 3 — Email Validation

Perform basic checks using `startswith()`, `endswith()`, `find()`, `rfind()`, and `count()`.

```
----- EMAIL ANALYSIS -----
Email : john.doe@gmail.com
Starts with user : True
Ends with .com : True
@ position : 8
Last . position : 14
Number of dots : 2
```

Part 4 — Skill Processing

Use `split()` to separate the skills and `join()` to create a clean display string. Convert the skills into a cleaner form using `title()`.

```
python, django, react, sql → Python | Django | React | Sql
```

Part 5 — Profile Card

Generate a profile card using f-strings.

```
=====
CODEHUB PROFILE
=====

Name : John Doe
Username : john.doe
Email : john.doe@gmail.com
Phone : 9876543210
City : Hyderabad
Technology : Python
Skills : Python | Django | React | Sql

=====
PROFILE CREATED!
=====
```

Part 6 — Developer ID

Create an ID using: first name + first character of city + favourite technology. Example: John + H + Python → JOHNHPYTHON. Use uppercase conversion, slicing, and an f-string.

Part 7 — Security Check

Given PyThOnDeVeLoPeR, use `swapcase()` to produce pYtHoNdEvElOpEr.

Part 8 — Case-Insensitive Technology Check

Ask the user what technology they are learning. Treat PYTHON, Python, python, and PyThOn as the same using `casefold()`.

Part 9 — Find Your Name

Ask for a character or piece of text from the user's name. Use `find()`, `rfind()`, and `count()` to report its first occurrence, last occurrence, and number of occurrences. Use `index()` for a character you know exists.

Part 10 — String Literal Challenge

Create a multi-line CodeHub welcome message using a triple-quoted string literal.

```
=====
WELCOME TO CODEHUB
=====

Learn Python.
Build APIs.
Create React applications.
Become a Full Stack Developer.
```

Final Challenge — Mini Full Stack Registration

Combine everything into one complete program. The flow should include registration, processing, profile display, email analysis, and developer information.

```
=====
CODEHUB USER REGISTRATION
=====

Enter your full name:
Enter your email:
Enter your phone number:
Enter your city:
```

```
Enter your favourite technology:
Enter your skills:
```

```
Processing profile...
```

```
=====
PROFILE
=====
```

```
Name :
Username :
Email :
Phone :
City :
Technology :
Skills :
```

```
=====
EMAIL ANALYSIS
=====
```

```
Starts with username :
Ends with .com :
@ position :
Last . position :
Number of dots :
```

```
=====
DEVELOPER INFORMATION
=====
```

```
Developer ID :
Security Key :
```

Bonus Challenges

Bonus 1 — Username Generator: Generate `john.doe` from `John Doe` without asking for a separate username.

Bonus 2 — Domain Extractor: From `john.doe@gmail.com`, extract `gmail.com`. Hint: `find()` + slicing.

Bonus 3 — First & Last Name: From `John Doe`, extract first name and last name using `split()`.

Bonus 4 — Skill Counter: Given `python, django, python, react, python, sql`, use `count()` to determine how many times `python` appears.

Bonus 5 — Clean Messy Input: Process inputs such as `" JOHN DOE "`, `" JOHN.DOE@GMAIL.COM "`, `" Hyderabad "`, and `" python, django, react "` to produce clean profile data.

Submission Goal: Submit one Python file containing your complete Mini Full Stack Registration program. Keep the code readable and add comments wherever you use a string method.